Car Hire Management System Design

Table of Contents

- Introduction
- Major Requirements
- System Components and Modules
- User Panels and Features
- Technical Architecture
- Database Design
- Security and UX Considerations
- Conclusion

↘ Introduction

A Car Hire Management System is a software solution that helps manage the day-to-day operations of a car rental business. It streamlines processes such as booking cars, tracking vehicles, handling payments, and managing customer data. This guide provides a comprehensive design for a Car Hire Management System using PHP, MySQL, XAMPP, and related web technologies. The system will be user-friendly and modular, with separate panels for administrators and customers, ensuring clarity and ease of use for all stakeholders.

↘ Major Requirements

Designing a robust car hire system requires addressing several key requirements:

- **Vehicle Management:** Maintain an inventory of available cars, including details like make, model, year, license plate, and status (available, rented, in maintenance). Provide functionality to add new vehicles and mark them as out of service or in maintenance.
- **Booking and Reservation:** Allow customers to search for available cars by date, time, and location. Enable online bookings with real-time availability checks to prevent double bookings. Customers should be able to view booking details, confirm reservations, and cancel or modify bookings within specified policies.

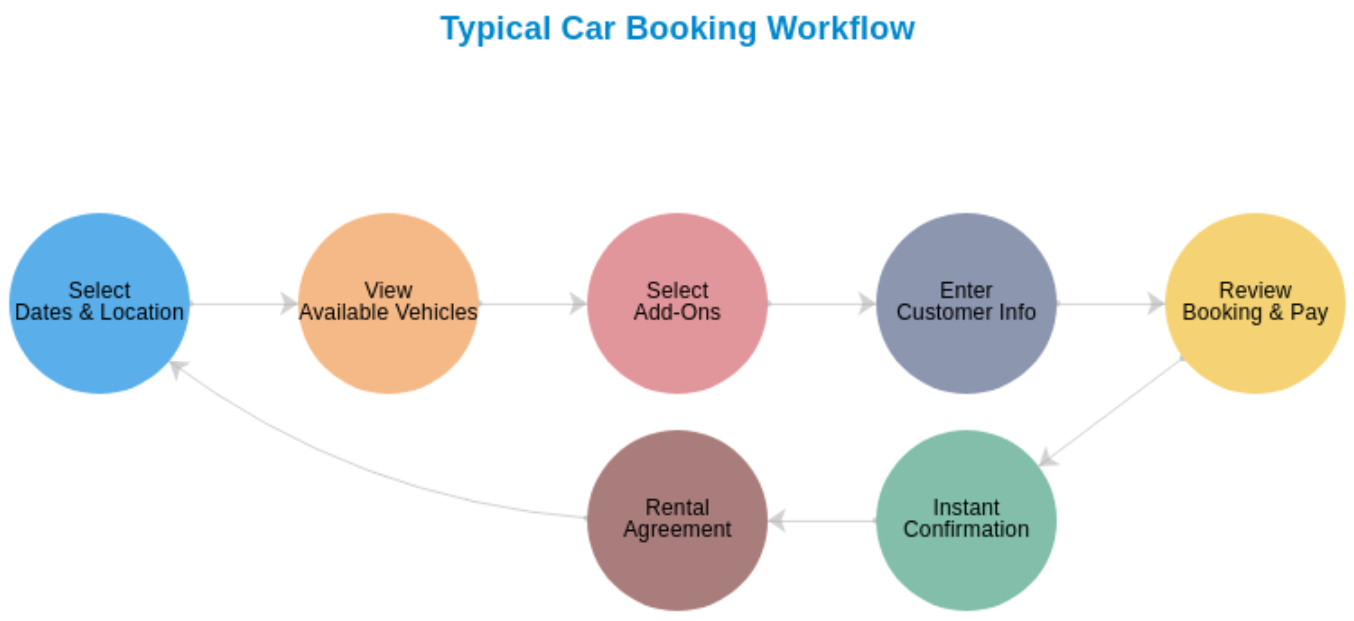
- **Customer Management:** Store customer information such as name, contact details, driver's license number, and rental history. Provide a way to register new customers and track their past rentals for loyalty programs or credit checks.
- **Payment Processing:** Integrate secure payment gateways to handle rental fees, deposits, and returns. Support multiple payment methods (credit/debit cards, mobile wallets, etc.) and issue receipts after each transaction. Ensure PCI-DSS compliance for handling cardholder data.
- **Fleet Tracking:** If vehicles are equipped with GPS, integrate a tracking system to monitor vehicle location and route in real time. This helps in dispatching vehicles, ensuring security, and tracking mileage for maintenance.
- **Maintenance and Service:** Schedule regular vehicle maintenance and service checks. The system should alert administrators when a vehicle is due for service or inspection. Keep a log of repairs and costs associated with each vehicle.
- **Reporting and Analytics:** Generate reports on key metrics such as daily/weekly/monthly revenue, popular vehicles, customer bookings, and occupancy rates. Provide dashboard analytics to help management make informed decisions. Reports can be exported to CSV or PDF for record-keeping.
- **Multilingual and Multi-Currency Support:** If the business serves international customers, support multiple languages and currencies. Ensure the system can handle different time zones and currency conversions for rental rates.
- **Mobile Responsiveness:** Ensure the system is accessible and functional on mobile devices, as a significant portion of users may access it via smartphones or tablets. This includes a responsive design and possibly a mobile app for users.
- **User Roles and Permissions:** Implement role-based access control. Different users (admin, manager, receptionist, customer) should have distinct permissions. For example, administrators can manage all aspects of the system, while customers can only view their own bookings.

↘ System Components and Modules

The Car Hire Management System is composed of several interconnected components and modules. Each module focuses on a specific set of functionalities:

- **Vehicle Management Module:** This module handles all operations related to the car fleet. It includes sub-modules for adding/editing vehicles, updating vehicle status (available, rented, in maintenance), and managing vehicle specifications. For example, when a car is rented, its status changes to "rented" in the system, and when it's returned, the status switches back to "available."
- **Booking Module:** The core of the system, the booking module manages the entire reservation lifecycle. It allows users to search for available cars on specific dates, create new bookings, and view booking details. This module ensures that when a customer books a car, the system checks availability and updates the booking status. It also generates booking confirmations and handles

modifications or cancellations. The following chart illustrates the typical flow of a car booking, from searching for a vehicle to finalizing the rental.



Data Source:

- **Customer Management Module:** This module deals with customer records and interactions. It allows adding new customers, updating their information, and retrieving customer details. It also tracks customer rental history (for loyalty programs or credit checks) and provides a search function to find specific customers. The system can flag repeat customers or those with past issues (like unpaid fees).

- **Payment Processing Module:** This module integrates with payment gateways to handle rental payments. It includes functions for taking payments (deposits or full payment at booking), processing refunds, and managing transactions. The system generates invoices or receipts for each payment and ensures that payments are linked to specific bookings. Security is paramount here, so all payment data is encrypted and PCI-compliant.

- **Fleet Tracking Module:** If the system is integrated with a GPS tracking system, this module provides real-time location data for vehicles. It can display a map view of all active rentals and allow tracking of a specific vehicle's route. This module is optional for a basic system but valuable for large fleets or for ensuring vehicle security.

- **Maintenance Module:** This module schedules and records vehicle maintenance. It allows creating service schedules (e.g., every 5,000 km or 6 months) and setting alerts for upcoming services. When a vehicle's maintenance is due, the system can notify the maintenance team. The module also logs maintenance records (service date, cost, parts replaced) for each vehicle.

- **Reporting and Analytics Module:** This module aggregates data from various parts of the system to produce reports. It can generate daily sales reports, monthly revenue reports, and reports on popular cars or peak booking times. The analytics dashboard might include charts and graphs for

quick insights. For instance, it could show occupancy rates over time or revenue by location. This module is crucial for business intelligence, helping management make strategic decisions.

- **Admin Panel Module:** The admin panel is a centralized control hub for the system. It provides an interface for administrators to manage all modules. This includes managing user roles, system settings (like rates, currency, language), and configuration of the system. The admin panel ensures that only authorized personnel can make changes to critical data.

These modules work together to provide a seamless experience. For example, when a customer books a car, the booking module checks availability via the vehicle management module, the payment module handles the payment, and the maintenance module might schedule the next service for that vehicle. Each module is designed to be independent enough to allow updates or changes without affecting others, ensuring maintainability and scalability.

↘ User Panels and Features

The system will have distinct user panels for different roles. Below is a description of each panel and its key features:

- **Customer Panel:** This is the user-facing part of the system, accessible to customers or clients who are booking cars. It includes the following features:
- **Vehicle Search:** Customers can search for available cars by entering pick-up and drop-off locations, dates, and times. The system displays a list of available cars for the selected period with details like model, price per day, and availability. Customers can filter results by car type or other criteria.
- **Booking Process:** Once a customer finds a suitable car, they can initiate a booking. The system shows the rental period, total cost, and any applicable taxes. The customer can enter their personal details (name, email, phone, driver's license number) and confirm the booking. Upon confirmation, the customer receives an email or SMS with the booking confirmation number and details.
- **Booking History:** Customers can view their past bookings, including details of each rental (car, dates, status). This helps them track previous rentals and ensure they have returned all vehicles. The system can also allow customers to cancel or modify a booking if within the allowed time frame.
- **Profile Management:** Customers can update their profile information (address, contact details, etc.) through the panel. They can also upload a copy of their driver's license for verification.
- **Customer Support:** A contact form or live chat option might be available for customers to reach out with queries or issues related to their bookings.
- **Admin Panel:** The admin panel is the backend interface for system administrators. It offers comprehensive control over all aspects of the car hire business. Key features include:
- **User Management:** Admins can add, edit, or delete user accounts (staff or customers) and assign roles and permissions. For example, they can create a new receptionist user with limited access to

bookings or create an admin user with full rights.

- **Vehicle Management:** Admins can add new vehicles to the inventory, set their specifications (model, year, color, capacity, etc.), and update their status. They can also mark a vehicle as out of service for maintenance or sell it from the system.
- **Booking Management:** Admins have visibility into all active bookings. They can view pending reservations, check which cars are rented, and handle any issues like double bookings or no-shows. Admins can manually create bookings for walk-in customers or adjust existing bookings if needed.
- **Customer Management:** Admins can search for any customer record, view their rental history, and update their details. This is useful for handling customer complaints or verifying information.
- **Payment Processing:** The admin panel can handle transactions that were not processed online (for example, if a customer pays in cash). Admins can log payments against bookings and issue refunds or credit notes if necessary.
- **Maintenance Scheduling:** Admins can schedule maintenance for vehicles or log past maintenance tasks. They can set up recurring maintenance reminders and view maintenance logs for each vehicle.
- **Reporting and Analytics:** The admin panel provides access to all reports generated by the system. Admins can run daily revenue reports, check occupancy rates, and analyze business performance. They can also export these reports for record-keeping or presentation.
- **System Settings:** Admins can configure system-wide settings such as rental rates, currency, language, and company information. This ensures the system reflects the current business policies.
- **Security Features:** The admin panel includes security features like login credentials management, password policies, and possibly two-factor authentication for admin users.
- **Staff Panel (Optional):** In larger operations, there might be a separate staff panel for employees such as receptionists or dispatchers. This panel could include features like checking vehicle availability, managing check-in/check-out for rented cars, and handling minor customer interactions. The staff panel is more limited than the admin panel, focusing on day-to-day operational tasks. For example, a receptionist might use the staff panel to check which cars are available for the day, print out rental agreements, and verify customer IDs.

Each panel is designed with user-friendliness in mind. The customer panel has a simple, intuitive interface for booking cars, while the admin panel provides robust controls with a clear navigation menu. This separation ensures that users only see the features relevant to their role, reducing clutter and making the system easier to use.

↘ Technical Architecture

The Car Hire Management System will be built using a web application architecture with the following components:

- **Front-End:** The user interface will be developed using HTML5, CSS3, and JavaScript. This will provide a responsive design that works on both desktop and mobile devices. JavaScript libraries (e.g., jQuery or modern frameworks like React, Vue.js, or Angular) can be used to enhance interactivity. For instance, JavaScript can be used to implement real-time availability checks when a customer selects dates on the booking form. The front-end will communicate with the server using AJAX requests to fetch data or perform actions without reloading the page.

- **Back-End:** The server-side logic will be implemented using PHP. PHP is a widely used server-side scripting language that is well-suited for web development and integrates smoothly with MySQL. The back-end will handle user authentication, process form submissions, interact with the database, and manage business logic. For example, when a customer submits a booking form, the PHP back-end will validate the input, check vehicle availability, and then either create a new booking record in the database or return an error if the booking is not possible. The use of a framework like Laravel or CodeIgniter could simplify development, but for this design, we'll assume a custom PHP implementation.

- **Database:** A MySQL database will be used to store all data. MySQL is a popular, open-source relational database management system that is reliable and well-supported. The database will contain tables for vehicles, customers, bookings, payments, maintenance records, and more. We will use SQL queries or an ORM (Object-Relational Mapping) tool in PHP to interact with the database. For example, to retrieve available cars for a given date, the system might run a SQL query that joins the `vehicles` table with the `bookings` table to exclude cars that are already rented during that period. Using an ORM can abstract the database interactions, making the code cleaner and more maintainable.

- **Web Server:** XAMPP (Cross-platform, Apache, MySQL, PHP, Perl) will be used as the local development environment. XAMPP provides a convenient way to set up a web server, database, and PHP environment on a local machine. In a production environment, a similar setup (e.g., Apache or Nginx web server with PHP and MySQL) will be used. The system will be deployed on a server with appropriate permissions to read/write to the database and handle file uploads (for things like customer ID uploads).

- **Communication:** The front-end and back-end will communicate using HTTP requests. Typically, form submissions will use POST requests to the server, and the server will respond with JSON data or HTML pages. For example, when a customer submits a booking form, the browser sends a POST request to a PHP script on the server. The PHP script processes the request, updates the database, and then either redirects the user to a confirmation page or returns a JSON object with the booking ID.

- **Security Measures:** The architecture will include security measures to protect the application. This includes using HTTPS for all communications (so that data is encrypted in transit), sanitizing and validating all user inputs to prevent SQL injection and other attacks, and implementing proper authentication and authorization. The back-end will check user roles before allowing certain actions (for example, only an admin can delete a vehicle record). Additionally, sessions will be used to manage user logins securely.

- **Third-Party Integrations:** The system will integrate with third-party services where necessary. For example, integrating with a payment gateway API (like Stripe or PayPal) will be done using their provided libraries or SDKs in PHP. If GPS tracking is used, the system might integrate with a telematics

API to fetch location data. These integrations will be implemented as separate modules or functions in the back-end, ensuring they are modular and can be updated independently.

- **Scalability:** The architecture is designed to be scalable. As the business grows, additional servers or services can be added. For instance, the database can be sharded or replicated for better performance, and the application can be deployed across multiple web servers behind a load balancer. Using PHP's ability to connect to databases and the stateless nature of HTTP requests means the system can handle multiple concurrent users without issues. Caching mechanisms (like Redis or Memcached) can be added to improve response times for frequently accessed data.

This technical architecture provides a solid foundation for the Car Hire Management System. It separates concerns (front-end, back-end, database), making it easier to develop and maintain each component independently. The use of open-source technologies (PHP, MySQL, XAMPP) keeps the solution cost-effective and widely supported.

↘ Database Design

A well-structured database is crucial for the system's performance and data integrity. The following is a proposed database schema with key tables and their relationships:

- **Customers Table:** Stores information about customers.
  - `customer_id` (Primary Key, Auto-Increment)
  - `first_name`
  - `last_name`
  - `email`
  - `phone`
  - `address`
  - `license_number` (Driver's license number)
  - `license_expiry` (Expiry date of driver's license)
  - `nationality`
  - `passport_number` (Optional, if needed for international customers)
  - `created_at`
  - `updated_at`
- **Vehicles Table:** Stores details about each vehicle in the fleet.
  - `vehicle_id` (Primary Key, Auto-Increment)
  - `make`

    - `model`
    - `year`
    - `color`
    - `license_plate`
    - `capacity` (Number of passengers)
    - `transmission` (Manual or Automatic)
    - `fuel_type`
    - `daily_rate` (Daily rental rate)
    - `weekly_rate` (Weekly rental rate)
    - `status` (Available, Rented, Maintenance, Sold) – Enum or String
    - `notes` (Any additional notes about the vehicle)
    - `created_at`
    - `updated_at`
  - **Bookings Table:** Records each rental booking.
    - `booking_id` (Primary Key, Auto-Increment)
    - `customer_id` (Foreign Key referencing Customers)
    - `vehicle_id` (Foreign Key referencing Vehicles)
    - `pickup_date`
    - `pickup_time`
    - `dropoff_date`
    - `dropoff_time`
    - `total_rent_days` (Calculated as difference between dropoff and pickup dates)
    - `total_amount` (Calculated rental cost)
    - `booking_date` (Date when the booking was made)
    - `status` (Confirmed, Pending, Cancelled, Completed) – Enum or String
    - `deposit_paid` (Boolean or amount paid as deposit)
    - `insurance_cover` (Boolean or type of insurance cover)
    - `additional_driver` (Boolean or number of additional drivers)
    - `created_at`

    - `updated_at`
- **Payments Table:** Tracks all financial transactions related to bookings.
    - `payment_id` (Primary Key, Auto-Increment)
    - `booking_id` (Foreign Key referencing Bookings)
    - `payment_date`
    - `amount`
    - `payment_method` (e.g., Credit Card, Cash, PayPal)
    - `payment_reference` (Transaction ID from payment gateway)
    - `is_paid` (Boolean, indicating if payment is complete)
    - `created_at`
    - `updated_at`
- **Maintenance Table:** Logs maintenance and service records for vehicles.
    - `maintenance_id` (Primary Key, Auto-Increment)
    - `vehicle_id` (Foreign Key referencing Vehicles)
    - `service_date`
    - `service_type` (e.g., Oil Change, Tire Rotation, Major Service)
    - `cost`
    - `notes`
    - `created_at`
    - `updated_at`
- **Admin Users Table:** Stores user accounts for the admin panel.
    - `user_id` (Primary Key, Auto-Increment)
    - `username`
    - `password` (hashed)
    - `email`
    - `first_name`
    - `last_name`
    - `role` (Admin, Manager, Receptionist) – Enum or String
    - `created_at`

- `updated_at`

Note: The above schema is a simplified version. Additional tables might be needed for features like **Add-Ons** (for optional extras like GPS or insurance), **Locations** (if multiple branch locations exist), or **Notifications** (for sending email/SMS alerts). Relationships between tables are established using foreign keys:

- The `Customers` table is linked to the `Bookings` table via `customer_id`, meaning each booking is associated with one customer.
- The `Vehicles` table is linked to the `Bookings` table via `vehicle_id`, meaning each booking is for one vehicle.
- The `Bookings` table is linked to the `Payments` table via `booking_id`, meaning each booking can have one or more payments (for example, a deposit and a final payment).
- The `Vehicles` table is linked to the `Maintenance` table via `vehicle_id`, meaning each vehicle can have multiple maintenance records.
- The `Admin Users` table is for authentication; it can be expanded with additional fields if needed (like a user's profile picture or department).

Database normalization is applied to avoid redundancy. For instance, rental rates are stored in the `Vehicles` table rather than the `Bookings` table to prevent storing the same rate multiple times. Indexes will be created on columns frequently used in queries (like `customer_id` in `Bookings` and `vehicle_id` in `Bookings`) to ensure fast lookups.

By designing the database in this way, the system can efficiently retrieve and update data. For example, to find all bookings for a specific customer, the system can join the `Bookings` and `Customers` tables. To check if a vehicle is available on a certain date, it can query the `Bookings` table to see if there are any existing bookings for that vehicle on that date. Proper indexing and normalization will ensure that these operations are fast even as the data volume grows.

⇲ Security and UX Considerations

Developing a secure and user-friendly Car Hire Management System involves implementing robust security measures and focusing on a great user experience. Here are key considerations:

- **Authentication and Authorization:** Implement a secure authentication system for all users. Use hashed passwords and consider multi-factor authentication (MFA) for admin accounts. Only authenticated users should access certain pages or perform certain actions. For example, a customer should only be able to view their own bookings, while an admin can manage all bookings. Role-based access control (RBAC) should be used so that permissions are assigned based on user roles (admin, manager, etc.). This ensures that even if an unauthorized user somehow accesses a page, they won't be able to perform actions beyond their allowed scope. As a best practice, implement role-based permissions to ensure users only access what they need.

- **Data Encryption:** All sensitive data should be encrypted. This includes customer personal information (names, addresses), driver's license numbers, and payment card details. Use strong encryption algorithms (e.g., AES) for data at rest (stored in the database) and ensure data in transit is encrypted via HTTPS. Additionally, consider encrypting database backups and using secure password hashing to protect user credentials. Encrypting data both at rest and in transit is a fundamental requirement for any modern software system.

- **Secure Payment Processing:** Integrate with a reputable payment gateway to handle transactions. Ensure the system is PCI-DSS compliant, which involves securely storing card data, using tokenization, and adhering to best practices for online payments. The system should never store full credit card numbers; instead, use tokens or payment gateway vaults. Also, validate and sanitize all input fields to prevent injection attacks. For example, any user-supplied input (like booking form fields) should be validated to ensure it's in the correct format before being used in database queries or sent to payment processors. In fact, it's advisable to avoid syncing personal devices to rental cars to mitigate security risks, just as you should avoid storing sensitive data in plain text.

- **User Interface (UI) Design:** Design an intuitive and easy-to-use interface. Use a clean layout with clear navigation. For the customer panel, the booking process should be straightforward, with step-by-step forms and clear instructions. For the admin panel, organize the interface logically (e.g., separate menus for Vehicles, Bookings, Customers, etc.). Use consistent design patterns and provide helpful error messages and confirmation dialogs. For instance, if a customer tries to book a car that's already rented, the system should display a clear error message instead of failing silently. A well-designed UI not only improves usability but can also reduce errors and increase user satisfaction.

- **Responsive Design:** Ensure the system is fully responsive. This means it should work on different screen sizes, from large desktop monitors to smartphones. Use responsive web design techniques (media queries, flexible grids) so that the layout adjusts automatically. Mobile users should be able to easily access the booking form and view their bookings on a small screen. A responsive design is essential in today's multi-device environment, as it improves accessibility and user experience.

- **Performance and Scalability:** Optimize the system for performance. This includes caching frequently accessed data (like popular vehicles or recent bookings) to reduce database load, using efficient SQL queries, and ensuring the web server and database can handle the expected number of concurrent users. If the system grows, it should be able to scale horizontally (adding more servers) or vertically (upgrading server resources) without significant downtime. Load testing and performance monitoring should be done to identify and fix bottlenecks early. Fast performance ensures that users don't experience long waits, which is crucial for a smooth booking process.

- **User Feedback and Error Handling:** Provide feedback to users for every action. For example, when a customer submits a booking, show a confirmation message or redirect to a confirmation page. If an error occurs (like a server issue or invalid input), display a friendly error message explaining the problem and possible solutions. Log errors securely and don't expose sensitive information in error messages. Good error handling and feedback improve user confidence and help users resolve issues quickly.

- **Accessibility:** Make the system accessible to users with disabilities. Use semantic HTML, provide alt text for images, and ensure forms are keyboard-navigable. This might include using ARIA roles for

dynamic content and ensuring color contrast meets accessibility standards. An accessible system ensures that all customers, including those with disabilities, can use the service effectively.

- **Compliance and Privacy:** Comply with relevant data protection regulations (like GDPR in Europe if serving European customers). This means obtaining user consent for data processing, allowing users to access or delete their data, and being transparent about data usage. Additionally, comply with any industry-specific regulations (for example, if dealing with rental car insurance, follow insurance industry standards). Keeping up with compliance requirements not only avoids legal issues but also builds trust with customers.
- **Continuous Monitoring and Updates:** Implement monitoring to detect any security incidents or performance issues. Regularly update the software with security patches and software updates. Use intrusion detection systems and security audits to ensure the system remains secure over time. Security is an ongoing process, so the system should be designed with updateability in mind.

By addressing these security and UX considerations, the Car Hire Management System will be both robust and user-friendly. A secure system protects customer data and business operations, while a user-friendly system increases customer satisfaction and efficiency for staff. This combination is essential for the long-term success of the car rental business.

↘ Conclusion

Designing a Car Hire Management System involves careful planning of requirements, system components, user interfaces, and technical architecture. By following the steps outlined in this guide – identifying major requirements, defining system modules, designing user panels, setting up the technical architecture, creating a solid database schema, and ensuring security and UX – you can build a comprehensive and efficient system. The system will enable your car rental business to manage bookings, vehicles, and customers effectively, all while providing a seamless experience for customers and robust control for administrators. With proper implementation and ongoing maintenance, this system will support your business growth and improve operational efficiency.

Reference

[1] Vehicle Booking & Management System
https://softhealer.com/blog/articals-11/the-ultimate-guide-to-vehicle-rental-management-systems-in-2025-12856

[2] Car Rental Reports & Business Intelligence Software
https://www.autycloud.com/Reports-Business-Intelligence-Car-Rental-Software

[3] Car Rental Software with Online Booking & Payments 2026
https://www.autycloud.com/blog/car-rental-software-online-booking-payments-2026

[4] PCI DSS - Compliance Guide
https://www.vikingcloud.com/blog/pci-dss-compliance-guide

[5] Car Rental Software Features, Modules & Capabilities [2026]
https://technologyevaluation.com/features-list/car-rental-software-61353?srsltid=AfmBOooaXRxsa4g_NYv6sy5-2N7f20lQZCBSehQEADOzcxNxHer8XS9t

[6] Development of Car Rental Management System with ...
https://www.researchgate.net/publication/350603452_Development_of_Car_Rental_Management_System_with_Scheduling_Algorithm

[7] Design of a Car Rental Application User Interface Using ...
https://ijsr.internationaljournallabs.com/index.php/ijsr/article/view/2874

[8] car-rental-system
https://github.com/topics/car-rental-system

[9] Database Design for Booking and Reservation Systems
https://www.geeksforgeeks.org/dbms/how-to-design-a-database-for-booking-and-reservation-systems/

[10] Car Rental Agency Database System
https://github.com/guptavasu1213/Car-Rental-Agency

[11] How to integrate a payment gateway into a mobile app
https://www.weareplanet.com/blog/integrate-payment-gateway-mobile-app

[12] Car Rental Software - Vehicle Rental Management & Booking ...
https://hqrentalsoftware.com/

[13] Development of Car Rental Management System with ...
https://www.ijset.in/wp-content/uploads/IJSET_V9_issue2_162.pdf

[14] User Authorization in SaaS: Models & Best Practices
https://frontegg.com/blog/user-authorization

[15] Entity-relationship diagram for car rental (MySQL)
https://stackoverflow.com/questions/13545752/entity-relationship-diagram-for-car-rental-mysql

[16] Database Design for Booking and Reservation Systems
https://www.geeksforgeeks.org/dbms/how-to-design-a-database-for-booking-and-reservation-systems/

[17] Cybersecurity in Car Rental: Modernizing Software to ...
https://www.loopit.co/en-us/blog/cybersecurity-in-car-rental-modernizing-software-to-protect-customer-data-and-build-trust

[18] Designing Parking Lot (Garage) System | System Design
https://www.geeksforgeeks.org/system-design/designing-parking-lot-garage-system-system-design/

[19] Guide to Building Audit Logs for Application Software
https://medium.com/@tony.infisical/guide-to-building-audit-logs-for-application-software-b0083bb58604

[20] What features are typically associated with user roles in the ...
https://www.vehicletracking.qa/software-help-support/what-features-are-typically-associated-with-user-roles-in-the-software

[21] Cybersecurity in Car Rental: Modernizing Software to ...
https://www.loopit.co/en-us/blog/cybersecurity-in-car-rental-modernizing-software-to-protect-customer-data-and-build-trust

[22] Big, hidden risk of syncing phone to rental car infotainment ...
https://www.cnbc.com/2024/09/25/hidden-risk-of-syncing-your-phone-to-rental-car-infotainment-system.html

[23] Car Rental Mobile App UI UX Design in 2025
https://medium.com/@prajapatisuketu/car-rental-mobile-app-ui-ux-design-in-2025-c920111f3840

[24] How to Build a Car Rental App: Steps, Use Cases & Cost
https://www.strivemindz.com/blog/how-to-build-car-rental-app/